

سوالات و پاسخ‌های میان‌ترم درس بازیابی اطلاعات

دانشگاه شهید مدنی آذربایجان - نیمسال دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

سوال ۱

با در نظر گرفتن لیست‌های زیر:

$w1 : \langle 1, 2, 3 \rangle$

$w2 : \langle 4, 5 \rangle$

$w3 : \langle 4, 5 \rangle$

برای پرس‌وجوی زیر ترتیب مناسب و بهینه پردازش پرس‌وجو چگونه است؟

$w1 \text{ AND } w2 \text{ AND } w3$

پاسخ:

برای بهینه‌سازی پردازش پرس‌و‌جوهایی که با عملگر AND ترکیب شده‌اند، باید لیست‌ها را بر اساس طول آن‌ها (فرکانس سند یا Document Frequency) به صورت صعودی مرتب کنیم. دلیل این کار این است که اشتراک دو لیست کوچک، یک لیست موقت کوچکتر تولید می‌کند و هزینه‌ی مقایسه‌های بعدی به شدت کاهش می‌یابد. با توجه به داده‌های مسئله:

- طول لیست $w1$ برابر ۳ است.
- طول لیست $w2$ برابر ۲ است.
- طول لیست $w3$ برابر ۲ است.

بنابراین بهترین ترتیب پردازش، ابتدا ادغام $w2$ و $w3$ و سپس ادغام نتیجه‌ی آن با $w1$ است:

$(w2 \text{ AND } w3) \text{ AND } w1$ یا $(w3 \text{ AND } w2) \text{ AND } w1$

سوال ۲

برای ادغام لیست پست‌های زیر چند مقایسه لازم است؟ لیست مقایسه‌ها را بنویسید. (توجه کنید که در لیست اول از اشاره‌گرهای پرس با طول ۸ استفاده شده است و DOC ID هایی که دارای اشاره‌گر پرس هستند به صورت پررنگ مشخص شده است.)
(توجه داشته باشید که D برابر با رقم آخر شماره دانشجویی شما می‌باشد)

List1: <3, 5, 9, 15, 24, 39, 60, (70+D), 81, 84, 89, 92, 96, 97, 100, 115> skip=8

List2: <2, 12, 75, 76, 77, 78, 89, 95, 97, 99, 100, 101>

پاسخ:

رقم آخر شماره دانشجویی ۴۰۱۱۸۳۳۲۳۰ برابر صفر است ($D = 0$). بنابراین عنصر پررنگ وسط در لیست اول عدد ۷۰ می‌باشد.

با توجه به فرض سوال، لیست‌ها به شکل زیر هستند و عناصر پررنگ، گره‌های پرش Skip pointers را نشان می‌دهند:

● L1: <3, 5, 9, 15, 24, 39, 60, 70, 81, 84, 89, 92, 96, 97, 100, 115>

● L2: <2, 12, 75, 76, 77, 78, 89, 95, 97, 99, 100, 101>

در الگوریتم ادغام، زمانی که عنصر یک لیست از لیست دیگر کوچکتر است، بررسی می‌کنیم که آیا گره جاری دارای اشاره‌گر پرش است یا خیر. اگر بود، هدف پرش را با عنصر لیست دیگر مقایسه می‌کنیم تا ببینیم پرش مجاز است یا نه (این بررسی خود یک مقایسه به شمار می‌رود).
روند ۲۷ مقایسه به شرح زیر است (اشاره‌گر لیست اول p_1 و لیست دوم p_2):

۱. مقایسه (۳ و ۲) $3 > 2 \rightarrow$ (عنصر ۲ دارای پرش به ۹۷ است)
۲. بررسی پرش (۳ و ۹۷) $97 \not\leq 3 \rightarrow$ پرش ناموفق، p_2 به ۱۲ می‌رود.
۳. مقایسه (۳ و ۱۲) $3 < 12 \rightarrow$ (عنصر ۳ دارای پرش به ۷۰ است)
۴. بررسی پرش (۳ و ۱۲) $70 \not\leq 12 \rightarrow$ پرش ناموفق، p_1 به ۵ می‌رود.
۵. مقایسه (۵ و ۱۲) $5 < 12 \rightarrow p_1$ به ۹ می‌رود.
۶. مقایسه (۹ و ۱۲) $9 < 12 \rightarrow p_1$ به ۱۵ می‌رود.
۷. مقایسه (۱۵ و ۱۲) $15 > 12 \rightarrow p_2$ به ۷۵ می‌رود.
۸. مقایسه (۱۵ و ۷۵) $15 < 75 \rightarrow p_1$ به ۲۴ می‌رود.
۹. مقایسه (۲۴ و ۷۵) $24 < 75 \rightarrow p_1$ به ۳۹ می‌رود.
۱۰. مقایسه (۳۹ و ۷۵) $39 < 75 \rightarrow p_1$ به ۶۰ می‌رود.
۱۱. مقایسه (۶۰ و ۷۵) $60 < 75 \rightarrow p_1$ به ۷۰ می‌رود.
۱۲. مقایسه (۷۰ و ۷۵) $70 < 75 \rightarrow$ (عنصر ۷۰ دارای پرش به ۱۱۵ است)
۱۳. بررسی پرش (۷۵ و ۱۱۵) $115 \not\leq 75 \rightarrow$ پرش ناموفق، p_1 به ۸۱ می‌رود.
۱۴. مقایسه (۷۵ و ۸۱) $81 > 75 \rightarrow p_2$ به ۷۶ می‌رود.
۱۵. مقایسه (۷۶ و ۸۱) $81 > 76 \rightarrow p_2$ به ۷۷ می‌رود.
۱۶. مقایسه (۷۷ و ۸۱) $81 > 77 \rightarrow p_2$ به ۷۸ می‌رود.
۱۷. مقایسه (۷۸ و ۸۱) $81 > 78 \rightarrow p_2$ به ۸۹ می‌رود.
۱۸. مقایسه (۸۱ و ۸۹) $81 < 89 \rightarrow p_1$ به ۸۴ می‌رود.
۱۹. مقایسه (۸۴ و ۸۹) $84 < 89 \rightarrow p_1$ به ۸۹ می‌رود.
۲۰. مقایسه (۸۹ و ۸۹) $\rightarrow$ برابرند (یافت شد). p_1 به ۹۲ و p_2 به ۹۵ می‌رود.
۲۱. مقایسه (۹۲ و ۹۵) $92 < 95 \rightarrow p_1$ به ۹۶ می‌رود.
۲۲. مقایسه (۹۶ و ۹۵) $96 > 95 \rightarrow p_2$ به ۹۷ می‌رود.
۲۳. مقایسه (۹۶ و ۹۷) $96 < 97 \rightarrow p_1$ به ۹۷ می‌رود.

۲۴. مقایسه (۹۷ و ۹۷) $\rightarrow$ برابرند (یافت شد). p_1 به ۱۰۰ و p_2 به ۹۹ می‌رود. (چون برابر بودند، مقایسه پرش روی ۹۷ انجام نمی‌شود).

۲۵. مقایسه (۹۹ و ۱۰۰) $p_2 \rightarrow 99 > 100 \rightarrow$ به ۱۰۰ می‌رود.

۲۶. مقایسه (۱۰۰ و ۱۰۰) $\rightarrow$ برابرند (یافت شد). p_1 به ۱۱۵ و p_2 به ۱۰۱ می‌رود.

۲۷. مقایسه (۱۰۱ و ۱۱۵) $p_2 \rightarrow 101 > 115 \rightarrow$ به پایان لیست می‌رسد.

مجموع مقایسه‌ها: ۲۷ مقایسه.

عناصر مشترک یافت‌شده: 89, 97, 100.

سوال ۳

در یک سیستم IR هر یک از موارد زیر برای یک پرس‌وجوی دلخواه چه تأثیری می‌تواند روی Precision و Recall داشته باشد؟ پاسخ خود را با ذکر مثال و دلیل بنویسید.

۱. انجام stemming و lemmatization

پاسخ:

به طور کلی هر دو روش سعی در کاهش کلمات به ریشه یا فرم پایه آن‌ها دارند.

- تأثیر روی فراخوانی (Recall): افزایش می‌یابد. از آنجایی که اشکال مختلف یک کلمه (مثل جمع، فرم‌های مختلف فعل) به یک ریشه یکسان تبدیل می‌شوند، اسناد بیشتری بازیابی خواهند شد. به عنوان مثال اگر کاربر کوثری operate را جستجو کند، به دلیل ریشه‌یابی، اسنادی که شامل operating, operation, operative هستند نیز بازیابی می‌شوند که این امر باعث می‌شود هیچ سند مرتبطی از قلم نیفتد.
 - تأثیر روی دقت (Precision): کاهش می‌یابد. به دلیل ادغام کلمات با ریشه یکسان اما معنای متفاوت، ممکن است اسناد نامرتبلی به کاربر نمایش داده شود. مثلاً کلمه operating (سیستم‌عامل در کامپیوتر) و operative (عملیاتی / پزشکی) در روش Stemming هر دو به ریشه oper تبدیل می‌شوند. این امر باعث می‌شود در جستجوی مقالات کامپیوتری، مقالات نامرتبلی نیز بازگردانده شوند و دقت افت کند.
- نکته: روش Lemmatization نسبت به Stemming به دلیل استفاده از فرهنگ لغت و یافتن فرم واقعی و معنایی کلمه، دقت (Precision) بالاتری دارد و آسیب کمتری به آن می‌زند.

سوال ۴

فاصله ویرایشی بین دو کلمه نام خود (مثلاً Ali) و کلمه name را با استفاده از روش Levenshtein محاسبه کنید. (توجه داشته باشید که در این روش تنها سه عمل درج، حذف و جابجایی امکان‌پذیر است).

پاسخ:

نام فرض شده: ali (برای سادگی محاسبات با حروف کوچک).

کلمه هدف: name.

در روش لوندشتین هزینه عملیات‌های حذف، درج و جایگزینی برابر ۱ در نظر گرفته می‌شود. ماتریس برنامه‌نویسی پویا برای این دو کلمه به شکل زیر تشکیل می‌شود:

i	l	a		
۳	۲	۱	۰	
۳	۲	۱	۱	n
۳	۲	۱	۲	a
۳	۲	۲	۳	m
۳	۳	۳	۴	e

مقدار سلول پایینی سمت راست ماتریس برابر با ۳ است. بنابراین فاصله ویرایشی ۳ می باشد.
مراحل تبدیل (name → ali):

۱. درج حرف n در ابتدای کلمه → nali (هزینه: ۱)

۲. حرف a با حرف a می شود → تغییر نمی کند (هزینه: ۰)

۳. جایگزینی حرف l با m → nami (هزینه: ۱)

۴. جایگزینی حرف i با e → name (هزینه: ۱)

مجموع هزینه ها: ۳ عملیات.

سوال ۵

درستی یا نادرستی عبارات زیر را مشخص کنید.

(a) ریشه گیری (stemming) باعث افزایش اندازه دیکشنری در یک شاخص معکوس می شود.

(b) استفاده از اشاره گرهای پرش باعث تسریع پردازش کوثری X OR Y می شود.

پاسخ:

(a) نادرست (غلط). ریشه گیری باعث می شود که شکل های مختلف یک کلمه به یک ریشه واحد (مانند play) تبدیل شوند. این کار تعداد کلمات یکتا (Vocabulary) را کاهش داده و در نتیجه باعث کاهش اندازه دیکشنری می شود.

(b) نادرست (غلط). اشاره گرهای پرش (Skip Pointers) صرفاً برای تسریع در محاسبه اشتراک (Intersection) و کوثری های AND کاربرد دارند. در پردازش کوثری OR، باید اجتماع دو لیست محاسبه شود، به این معنی که تمامی عناصر هر دو لیست باید پیمایش و ادغام شوند، بنابراین پرش از روی عناصر امکان پذیر نیست و تأثیری در تسریع آن ندارد.

فرض کنید یک سیستم بازیابی اطلاعات با دیکشنری زیر داده شده است:

Term	ID Term
retried	۱
removed	۲
relieved	۳
retrieved	۴
redirected	۵
remained	۶

الف) برای پاسخ به پرس و جوی *iev* با استفاده از تریگرام‌ها (3-grams) چه مرحله‌ای باید طی شود؟ و چه کلماتی از لیست پاسخ این کوئری است؟ پاسخ خود را دقیق بنویسید.

پاسخ:

مراحل پردازش کوئری *iev*:

۱. تحلیل کوئری: از آنجایی که در ابتدا و انتهای کوئری کاراکتر * وجود دارد، ما نیازی به اضافه کردن علامت شروع و پایان (\$) نداریم. بنابراین تنها 3-gram موجود در این کوئری رشته iev است.
۲. استخراج تریگرام‌ها و جستجو در شاخص: در شاخص 3-gram، به دنبال posting list مربوط به کلید iev می‌گردیم. با بررسی کلمات دیکشنری می‌بینیم که کدام کلمات شامل iev هستند:

• 1: retried → ندارد

• 2: removed → ندارد

• 3: relieved → دارای edievrel

• 4: retrieved → دارای edievretr

• 5: redirected → ندارد

• 6: remained → ندارد

بنابراین لیست نامزدهای اولیه، اسناد (کلمات) با شناسه ۳ و ۴ خواهد بود.

۳. پس‌تصفیه (Post-filtering): کلمات بازیابی شده در مرحله قبل را واکشی کرده و به صورت رشته‌ای با عبارت منظم کوئری (در اینجا تطابق با زیررشته iev) چک می‌کنیم. از آنجا که هر دو کلمه relieved و retrieved دقیقاً شامل iev هستند، از این فیلتر عبور می‌کنند.

نتیجه نهایی: کلمات relieved (شناسه ۳) و retrieved (شناسه ۴) پاسخ این کوئری می‌باشند.

در روش ادغام لگاریتمی اگر اندازه شاخص کمکی درون حافظه‌ای $Z_0 = 8$ باشد و شاخص‌های I_3, I_2, I_1, I_0 را داشته باشیم. با فرض اینکه تعداد نشانه‌های (Token) ۶۵ باشد، شبیه‌سازی گام به گام الگوریتم ادغام لگاریتمی را اجرا کنید.

پاسخ:

ظرفیت حافظه موقت $Z_0 = 8$ توکن است. تعداد کل توکن‌ها $N = 65$ می‌باشد.

ظرفیت سطوح مختلف شاخص در ادغام لگاریتمی به شرح زیر است:

• ظرفیت $I_0 = 1 \times Z_0 = 8$

• ظرفیت $I_1 = 2 \times Z_0 = 16$

• ظرفیت $I_2 = 4 \times Z_0 = 32$

• ظرفیت $I_3 = 8 \times Z_0 = 64$

ما $65 \div 8 = 8$ به ۸ بلوک کامل (هر کدام ۸ توکن) داریم و ۱ توکن اضافه باقی می‌ماند. نمایش باینری عدد ۸ به صورت ۱۰۰۰ است که نشان می‌دهد در نهایت تنها سطح I_3 پر خواهد بود و بقیه سطوح دیسک خالی می‌مانند. شبیه‌سازی گام به گام:

۱. توکن‌های ۱ تا ۸: حافظه Z_0 پر می‌شود. چون I_0 خالی است، Z_0 به دیسک منتقل شده و I_0 ایجاد می‌شود (سایز ۸). Z_0 خالی می‌شود.

۲. توکن‌های ۹ تا ۱۶: حافظه Z_0 پر می‌شود. I_0 از قبل پر است. پس Z_0 و I_0 با هم ادغام شده و I_1 را تشکیل می‌دهند (سایز ۱۶). I_0 و Z_0 خالی می‌شوند.

۳. توکن‌های ۱۷ تا ۲۴: حافظه Z_0 پر می‌شود. I_0 خالی است، پس در I_0 نوشته می‌شود (سایز ۸).

۴. توکن‌های ۲۵ تا ۳۲: حافظه Z_0 پر می‌شود. I_0 پر است. ادغام Z_0 و I_0 رخ می‌دهد، نتیجه باید به I_1 برود. اما I_1 نیز پر است. پس Z_0, I_0, I_1 با هم ادغام شده و I_2 تشکیل می‌شود (سایز ۳۲). Z_0, I_0, I_1 همگی خالی می‌شوند.

۵. توکن‌های ۳۳ تا ۴۰: Z_0 پر می‌شود. I_0 ساخته می‌شود (سایز ۸).

۶. توکن‌های ۴۱ تا ۴۸: Z_0 پر می‌شود. I_0 ادغام شده و I_1 تشکیل می‌شود (سایز ۱۶).

۷. توکن‌های ۴۹ تا ۵۶: Z_0 پر می‌شود. I_0 ساخته می‌شود (سایز ۸).

۸. توکن‌های ۵۷ تا ۶۴: Z_0 پر می‌شود. I_0, I_1, I_2 همگی پر هستند. تمام این‌ها (Z_0, I_0, I_1, I_2) با هم ادغام آبشاری می‌شوند و I_3 تشکیل می‌شود (سایز ۶۴). تمامی سطوح پایین‌تر خالی می‌شوند.

۹. توکن ۶۵: وارد حافظه موقت Z_0 می‌شود.

وضعیت نهایی سیستم: I_3 دارای ۶۴ توکن (پر)، I_2 خالی، I_1 خالی، I_0 خالی و حافظه‌ی Z_0 دارای ۱ توکن خواهد بود.